

DEOS Core Architecture & Protocol Rules

Sürüm 0.3

DEOS – Dokuz Eylül Otonom Sistemler

Doküman tipi:	Interface Control Document (ICD)
Protokol sürümü:	v1.1 (0x11)
Taşıma altyapısı:	CAN-FD ana omurga
Durum:	ICD Sürüm 0.3 – v1.1 wire-format revizyonu

1 Amaç ve Kapsam

Bu doküman, DEOS araç içi CAN-FD haberleşme hattında kullanılan ortak adresleme, 29-bit Extended CAN Identifier yapısı, mesaj sınıfları, servisler, komut namespace'leri, ortak veri alanı, veri tipleri ve temel mesaj formatlarını tanımlar.

Dokümanın kapsamı CAN hattında gözlenebilen protokol davranışlarıdır. Node ID, Priority, Message Class, Service, Source, Destination, Command, Protocol Version, Sequence, Length ve Command Payload alanlarının wire-format karşılıkları normatif olarak tanımlanır.

Gömülü yazılım mimarisi ve CAN-FD hattı dışındaki taşıma ortamlarının çalışma ayrıntıları bu ICD'nin kapsamı dışındadır.

Bir Node ID fiziksel bir mikrodenetleyiciye birebir bağlı olmak zorunda değildir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal DEOS Node ID bulunabilir; ancak CAN hattında her Node ID bağımsız bir protokol kimliği olarak değerlendirilir.

2 Namespace Hiyerarşisi

DEOS protokolünde bir mesajın kimliği tek bir “mesaj numarası” ile değil, birden fazla alanın birleşimiyle belirlenir:

```
DEOS_CAN_PROTOCOL
|
+-- NODE
|   +-- SOURCE_NODE
|   +-- DESTINATION_NODE
|
+-- MESSAGE_CLASS
|
+-- SERVICE
|   +-- COMMAND
|       +-- PAYLOAD
|           +-- PARAMETER_ID    (gerektiğinde)
|           +-- VALUE           (gerektiğinde)
|
+-- PROTOCOL_METADATA
    +-- VERSION
    +-- SEQUENCE_RESERVED
    +-- LENGTH
```

Buna göre bir mesajın semantik kimliği genel olarak şu bileşenlerden oluşur:

Priority + Message Class + Service + Destination + Source + Command

Parametre yönetimi yapılan mesajlarda buna ek olarak **Parameter ID** kullanılır.

3 CAN-FD Extended Identifier Yapısı

DEOS protokolü 29-bit Extended CAN Identifier kullanır. Identifier alanı aşağıdaki şekilde bölünür:

Bitler	Alan	Uzunluk	Açıklama
28–26	Priority	3 bit	Arbitration önceliği
25–22	Message Class	4 bit	Mesaj davranış sınıfı
21–16	Service ID	6 bit	Fonksiyonel servis alanı
15–8	Destination Node	8 bit	Hedef düğüm adresi
7–0	Source Node	8 bit	Kaynak düğüm adresi

Toplam uzunluk:

$$3 + 4 + 6 + 8 + 8 = 29 \text{ bit}$$

Identifier sayısal olarak aşağıdaki bağıntı ile ifade edilir:

$$\text{CAN_ID} = (P \ll 26) | (C \ll 22) | (S \ll 16) | (D \ll 8) | \text{SRC}$$

Burada:

[nosep] P : Priority, C : Message Class, S : Service ID, D : Destination Node, SRC : Source Node değeridir.

CAN arbitration mantığı nedeniyle sayısal olarak daha düşük identifier daha yüksek önceliğe sahiptir. Bu nedenle Priority alanında düşük sayısal değer daha yüksek önceliği temsil eder.

3.1 Önerilen Priority Registry

Değer	Symbol	Kullanım
0	DEOS_PRIO_EMERGENCY	Acil durdurma ve en kritik safety mesajları
1	DEOS_PRIO_CRITICAL_CONTROL	Kritik gerçek zamanlı kontrol
2	DEOS_PRIO_CONTROL	Normal sürüş kontrol mesajları
3	DEOS_PRIO_STATUS	Durum ve geri besleme mesajları
4	DEOS_PRIO_CONFIG	Konfigürasyon ve kalibrasyon
5	DEOS_PRIO_NETWORK	Network management / heartbeat
6	DEOS_PRIO_LOW	Düşük öncelikli servis trafiği
7	DEOS_PRIO_BACKGROUND	Arka plan / geliştirme trafiği

4 Node Address Registry

Her fiziksel veya sanal DEOS haberleşme düğümüne sistem genelinde benzersiz bir 8-bit Logical Node ID atanır. Node ID, düğümün fiziksel donanım adresi veya transport adresi değildir; protokol içerisindeki mantıksal kimliğidir.

4.1 Adres Aralıkları

Adres Aralığı	Kullanım
0x00	Invalid / Unassigned
0x01–0x0F	Merkezi sistem ve virtual node'lar
0x10–0x1F	Aktüatör / motor ECU düğümleri
0x20–0x2F	Güç sistemi / BMS düğümleri
0x30–0x3F	Gateway / protocol bridge düğümleri
0x40–0x5F	Sensör düğümleri için ayrılmış alan
0x60–0xDF	Gelecek genişleme
0xE0–0xEF	Geliştirme / test düğümleri
0xF0–0xFE	Diagnostik ve servis araçları
0xFF	Broadcast

4.2 Virtual Node Atamaları

Virtual node, bağımsız bir fiziksel mikrodenetleyici veya bilgisayar olmak zorunda olmayan, ancak DEOS protokolünde kendi Source ve Destination Node ID'si ile adreslenebilen mantıksal haberleşme ucudur.

Node ID	Symbol	Çalıştığı Ortam	Görev
0x01	DEOS_NODE_TEXTUAL	Raspberry Pi / Textual	Operatör arayüzü; config, command, request/response, status ve log sink
0x02	DEOS_NODE_GROUND_CONTROL	Uzak yer kontrol uygulaması	Yer kontrol haberleşme ucu
0x04	DEOS_NODE_MICRO_ROS	Raspberry Pi / micro-ROS	ROS arayüzü, telemetry ve control entegrasyonu

Textual Node yalnızca konfigürasyon amacıyla kullanılan bir node değildir. Protokolün izin verdiği servislerde komut gönderebilir, cevap alabilir, durum bilgilerini işleyebilir ve log mesajlarının doğrudan hedefi olabilir. Benzer şekilde micro-ROS ve Ground Control Node'ları da ihtiyaç duydukları servisleri kaynak veya hedef olarak kullanabilir.

4.3 Physical Node Atamaları

Node ID	Symbol	Fiziksel Cihaz	Görev
0x03	DEOS_NODE_MAIN_STM32	STM32H563ZI	Ana fiziksel kontrol düğümü
0x10	DEOS_NODE_TRACTION	STM32 Traction ECU	Tahrik sistemi kontrolü
0x11	DEOS_NODE_STEERING	STM32 Steering ECU	Direksiyon sistemi kontrolü
0x12	DEOS_NODE_BRAKE	STM32 Brake ECU	Fren sistemi kontrolü
0x20	DEOS_NODE_BMS_MAIN	DALY BMS #1	Ana batarya BMS
0x21	DEOS_NODE_BMS_AUX	DALY BMS #2	Yardımcı batarya BMS
0x30	DEOS_NODE_BMS_GATEWAY	CAN Gateway	Classic CAN–CAN-FD çevirici

4.4 Servis ve Özel Node Atamaları

Node ID	Symbol	Tür	Görev
0xF0	DEOS_NODE_DIAG_TOOL	Servis aracı	Diagnostik ve servis erişimi
0xFF	DEOS_NODE_BROADCAST	Mantıksal	Tüm düğümlere yayın

4.5 Node Enum Tanımı

```

1 typedef enum
2 {
3     DEOS_NODE_INVALID          = 0x00 ,
4
5     DEOS_NODE_TEXTUAL          = 0x01 ,
6     DEOS_NODE_GROUND_CONTROL  = 0x02 ,
7     DEOS_NODE_MAIN_STM32       = 0x03 ,
8     DEOS_NODE_MICRO_ROS        = 0x04 ,
9
10    DEOS_NODE_TRACTION          = 0x10 ,
11    DEOS_NODE_STEERING          = 0x11 ,
12    DEOS_NODE_BRAKE             = 0x12 ,
13
14    DEOS_NODE_BMS_MAIN          = 0x20 ,
15    DEOS_NODE_BMS_AUX           = 0x21 ,
16
17    DEOS_NODE_BMS_GATEWAY       = 0x30 ,
18
19    DEOS_NODE_DIAG_TOOL         = 0xF0 ,
20    DEOS_NODE_BROADCAST         = 0xFF
21 } deos_node_id_t;

```

5 Message Class Registry

Message Class alanı mesajın hangi amaçla gönderildiğini tanımlar. Message Class, fonksiyonel servisten bağımsızdır.

ID	Symbol	Açıklama
0x0	DEOS_CLASS_SAFETY	Acil / safety-critical mesaj
0x1	DEOS_CLASS_COMMAND	Normal gerçek zamanlı kontrol komutu
0x2	DEOS_CLASS_STATUS	Durum / geri besleme / telemetri
0x3	DEOS_CLASS_EVENT	Fault veya olay bildirimi
0x4	DEOS_CLASS_REQUEST	Cevap beklenen okuma, sorgulama veya işlem isteği
0x5	DEOS_CLASS_RESPONSE	Bir REQUEST veya cevap gerektiren işleme verilen protokol cevabı
0x6	DEOS_CLASS_CONFIG	Parametre ve konfigürasyon işlemleri
0x7	DEOS_CLASS_NETWORK	Ağ yönetimi / heartbeat
0x8–0xF	Reserved	Gelecek kullanım

```
1 typedef enum
2 {
3     DEOS_CLASS_SAFETY    = 0x0,
4     DEOS_CLASS_COMMAND  = 0x1,
5     DEOS_CLASS_STATUS   = 0x2,
6     DEOS_CLASS_EVENT    = 0x3,
7     DEOS_CLASS_REQUEST  = 0x4,
8     DEOS_CLASS_RESPONSE = 0x5,
9     DEOS_CLASS_CONFIG   = 0x6,
10    DEOS_CLASS_NETWORK   = 0x7
11 } deos_message_class_t;
```

5.1 Message Class ve Service Arasındaki Fark

Message Class ile Service farklı sorulara cevap verir:

Alan	Cevapladığı soru	Örnek
Message Class	Mesaj node içerisinde hangi tür işlem akışına aittir?	COMMAND, CONFIG, NETWORK, RESPONSE
Service	Mesaj hangi fonksiyonel alana aittir?	STEERING, BRAKE, BMS, SYSTEM
Command	Bu fonksiyonel alan içerisinde tam olarak hangi işlem yapılacaktır?	SET_TARGET_ANGLE, SET_PARAMETER, PING

Aynı Service farklı Message Class'lar altında kullanılabilir. Örneğin STEERING servisi için:

```
COMMAND / STEERING / SET_TARGET_ANGLE
CONFIG   / STEERING / SET_PARAMETER
REQUEST  / STEERING / GET_STATUS
```

Burada Service her üç mesajda da direksiyon fonksiyonel alanını belirtirken Message Class, mesajların node içerisinde farklı işlem yollarına ayrılmasını sağlar.

Tersi de mümkündür: aynı Message Class farklı Service'lerde kullanılabilir:

```
COMMAND / TRACTION / SET_TARGET_SPEED
COMMAND / STEERING / SET_TARGET_ANGLE
COMMAND / BRAKE     / SET_TARGET
```

Bu örneklerde Message Class tüm mesajların normal kontrol komutu olduğunu, Service ise hangi alt sisteme ait olduklarını belirtir.

6 Service Registry

Service alanı mesajın hangi fonksiyonel domaine ait olduğunu gösterir.

Service ID	Symbol	Amaç
0x00	DEOS_SERVICE_SYSTEM	Genel node state/mode/reset bilgileri
0x01	DEOS_SERVICE_TRACTION	Tahrik sistemi
0x02	DEOS_SERVICE_STEERING	Direksiyon sistemi
0x03	DEOS_SERVICE_BRAKE	Fren sistemi
0x04	DEOS_SERVICE_BMS	Batarya ve BMS verileri
0x05	DEOS_SERVICE_CALIBRATION	Kalibrasyon işlemleri
0x06	DEOS_SERVICE_DIAGNOSTIC	Fault / sağlık / diagnostik
0x07–0x3F	Reserved	Yeni servisler için ayrılmış alan

```
1 typedef enum
2 {
3     DEOS_SERVICE_SYSTEM      = 0x00 ,
4     DEOS_SERVICE_TRACTION    = 0x01 ,
5     DEOS_SERVICE_STEERING    = 0x02 ,
6     DEOS_SERVICE_BRAKE       = 0x03 ,
7     DEOS_SERVICE_BMS         = 0x04 ,
8     DEOS_SERVICE_CALIBRATION = 0x05 ,
9     DEOS_SERVICE_DIAGNOSTIC  = 0x06
10 } deos_service_id_t ;
```

7 Node–Service İlişkisi

Node ve Service aynı kavram değildir. Node, fiziksel veya sanal haberleşme düğümünü; Service ise mesajın ait olduğu fonksiyonel alanı tanımlar. Bir node'un belirli bir servise mesaj gönderebilmesi, o servisin kendisi üzerinde barındırıldığı anlamına gelmez.

Aşağıdaki tablo temel service handler sahipliğini gösterir. Her adreslenebilir DEOS node'u, ayrıca bu ICD'de tanımlanan ortak SYSTEM/NETWORK PING-PONG davranışını desteklemek zorundadır.

Node	SYS	TRAC	STEER	BRAKE	BMS	CAL	DIAG
TEXTUAL	X	–	–	–	–	–	X
GROUND_CONTROL	X	–	–	–	–	–	X
MAIN_STM32	X	X	X	X	X	X	X
MICRO_ROS	X	–	–	–	–	–	X
TRACTION	X	X	–	–	–	X	X
STEERING	X	–	X	–	–	X	X
BRAKE	X	–	–	X	–	X	X
BMS_MAIN	X	–	–	–	X	–	X
BMS_AUX	X	–	–	–	X	–	X
BMS_GATEWAY	X	–	–	–	X	–	X

Tablodaki işaretler, ilgili node'un o servise ait mesajları desteklediğini ifade eder; mesaj gönderme yeteneğini sınırlamaz. Örneğin DEOS_NODE_TEXTUAL, hedefi DEOS_NODE_STEERING olan ve DEOS_SERVICE_STEERING kullanan bir CONFIG veya REQUEST mesajı gönderebilir.

Örneğin direksiyon düğümüne normal sürüş komutu gönderilirken **Destination** = DEOS_NODE_STEERING, **Service** = DEOS_SERVICE_STEERING kullanılır. Aynı fiziksel düğüme kalibrasyon komutu gönderilirken **Destination** = DEOS_NODE_STEERING, **Service** = DEOS_SERVICE_CALIBRATION kullanılır.

8 CAN-FD DLC, Payload Uzunluğu ve BRS

CAN-FD DLC alanı, frame'in fiziksel CAN-FD data field boyutunu belirtir. DEOS v1.1 Length alanı ise yalnızca geçerli command-specific payload byte sayısını belirtir. Bu iki alan aynı anlamda değildir.

8 byte üzerindeki CAN-FD data alanları 12, 16, 20, 24, 32, 48 veya 64 byte fiziksel boyutlara yuvarlanabilir. Bu nedenle DEOS Length değeri fiziksel CAN-FD data field boyutundan bağımsız olarak taşınır.

Örnek olarak 10 byte command payload için:

```
DEOS common header = 4 byte
Command payload     = 10 byte
Semantic data size  = 14 byte
CAN-FD data size    = 16 byte
Padding             = 2 byte
```

DLC yuvarlaması nedeniyle semantic data sonrasında kalan kullanılmayan byte'lar 0x00 olarak gönderilir ve alıcı tarafından payload'ın parçası olarak değerlendirilmez.

DEOS CAN-FD frame'leri 29-bit Extended Identifier, CAN-FD formatı ve Bit Rate Switching (BRS) kullanır. Nominal/arbitration bitrate ile data-phase bitrate değerleri deployment parametreleridir ve bu ICD revizyonunda sabitlenmemiştir.

9 REQUEST-RESPONSE ve CONFIG Ayrımı

REQUEST, RESPONSE ve CONFIG birbirinden bağımsız mesaj sınıflarıdır ve farklı amaçlara hizmet eder.

- **REQUEST:** Bir düğümden veri, durum veya belirli bir işlem sonucu talep etmek için kullanılır.
- **RESPONSE:** REQUEST sınıfındaki bir mesaja veya cevap gerektiren başka bir işleme karşılık dönen cevabı taşır.
- **CONFIG:** Bir düğümün çalışma parametrelerini, ayarlarını veya kalıcı yapılandırmasını değiştirmeye yönelik mesajları taşır.

Örneğin bir parametreyi okumak için REQUEST, aynı parametreyi değiştirmek için CONFIG kullanılabilir:

```
REQUEST / STEERING / GET_PARAMETER / PID_KP
CONFIG  / STEERING / SET_PARAMETER / PID_KP
```

Her iki işlem için de gerektiğinde RESPONSE sınıfı ile sonuç döndürülebilir. Bu ayrım, mesajın “cevap bekleyip beklemediği” ile “konfigürasyon amacı taşıyıp taşımadığı” bilgilerinin birbirine karıştırılmasını önler.

10 Response Altyapısı

RESPONSE Message Class protokolde ayrılmıştır; ancak hangi komutların zorunlu response üreteceği bu registry dokümanında kesinleştirilmemiştir.

Özellikle aşağıdaki işlemler gelecekte response kullanılabilecek adaylardır:

[nosep]Parameter GET/SET, state veya mode değişimi, kalibrasyon başlatma / kaydetme, fault reset / clear, node reset, servis tipi tek-seferlik yönetim komutları.

10.1 Generic Response Payload

DEOS_CLASS_RESPONSE mesajları varsayılan olarak standart bir veri yerleşimi kullanır. Eğer ilgili komut özel bir ek veri döndürmüyorsa, yanıt mesajının içeriği her zaman şu formatta olur:

Byte	Alan	Tip	Açıklama
0	Result Code	uint8_t	İşlem sonucu (deos_result_t)
1..N	Ek Veri (Opsiyonel)	Değişken	Komuta özgü veri

Result code 0x00 (SUCCESS) olduğunda işlem başarılı sayılır. Bu genel yapı sayesinde, her komut için özel bir response formatı tasarlamaya gerek kalmadan hata yönetimi standartlaştırılmıştır.

Aşağıda standart response result kodları listelenmiştir:

Kod	Anlam
0x00	SUCCESS
0x01	UNSUPPORTED
0x02	INVALID_PARAMETER
0x03	OUT_OF_RANGE
0x04	NOT_ALLOWED
0x05	BUSY
0x06	NOT_READY
0x07	INTERNAL_ERROR

Bu alanın kesin payload yerleşimi sonraki ICD revizyonunda mesaj bazında ayrıntılandırılacaktır.

11 Mantıksal Node Kimliği Kuralı

CAN hattında her Source ve Destination Node ID bağımsız bir DEOS protokol kimliğidir. Aynı fiziksel cihaz üzerinde birden fazla mantıksal Node ID bulunabilse bile, bir mesaj hangi Node ID adına üretiliyorsa CAN Identifier içerisindeki Source alanında o Node ID kullanılmalıdır.

Bir node başka bir node'un kimliğiyle cevap üretmemelidir. Örneğin `Destination = DEOS_NODE_BMS_MAIN` olan geçerli bir PING'e verilen PONG mesajında `Source = DEOS_NODE_BMS_MAIN` kullanılmalıdır.

Bu kural fiziksel yerleşimden bağımsızdır; CAN hattındaki diğer katılımcılar mantıksal Node ID'leri bağımsız haberleşme uçları olarak değerlendirir.

12 İsimlendirme Standardı

Tüm protokol sabitlerinde ortak prefix yapısı kullanılır:

```
DEOS_NODE_...
DEOS_PRIO_...
DEOS_CLASS_...
DEOS_SERVICE_...
DEOS_CMD_<SERVICE>_...
DEOS_PARAM_<SERVICE>_...
DEOS_STATE_...
DEOS_MODE_...
```

Örnekler:

```
DEOS_NODE_TEXTUAL
DEOS_NODE_GROUND_CONTROL
DEOS_NODE_MAIN_STM32
DEOS_NODE_MICRO_ROS
DEOS_NODE_STEERING
DEOS_CLASS_CONFIG
DEOS_SERVICE_STEERING
DEOS_CMD_STEERING_SET_TARGET_ANGLE
DEOS_CMD_SET_PARAMETER
DEOS_PARAM_STEERING_PID_KP
DEOS_STATE_ACTIVE
DEOS_MODE_AUTONOMOUS
```

13 Özet Registry

Mevcut CAN-FD protokol kararları aşağıdaki şekilde özetlenebilir:

- Ana araç içi haberleşme hattı CAN-FD'dir.
- 29-bit Extended CAN Identifier kullanılır.
- CAN-FD frame'lerinde BRS kullanılır.
- Priority, Message Class, Service, Destination ve Source alanları CAN Identifier içerisinde taşınır.
- Node ID, sistem genelinde kullanılan 8-bit mantıksal protokol kimliğidir.
- Aynı fiziksel cihaz birden fazla mantıksal Node ID barındırabilir; CAN hattında her biri bağımsız kimliktir.
- DEOS Protocol v1.1 sürüm değeri 0x11'dir.
- Ortak DEOS data header'ı Version, Sequence, Command ve Length olmak üzere 4 byte'tır.
- Command-specific maksimum payload uzunluğu 60 byte'tır.
- Length, yalnızca geçerli command-specific payload byte sayısını belirtir.
- DLC fiziksel CAN-FD data field boyutunu; Length ise semantic DEOS payload boyutunu ifade eder.
- DLC yuvarlaması sonucu oluşan kullanılmayan byte'lar 0x00 padding olarak gönderilir.
- Uygulama seviyesinde ek CRC veya ayrıca sync header kullanılmaz.
- Sequence alanı Source Node başına kullanılır; v1.1'de gap/duplicate kontrolü zorunlu değildir.
- REQUEST, RESPONSE ve CONFIG birbirinden ayrı Message Class değerleridir.
- CONFIG ayrı bir Service değildir.
- Parameter ID 16-bit'tir ve Service namespace'i içinde anlam kazanır.
- Varsayılan çok baytlı wire byte sırası Little-Endian'dır.
- Fiziksel değerlerde varsayılan yaklaşım ölçekli integer kullanımıdır.
- PING/PONG, DEOS_CLASS_NETWORK / DEOS_SERVICE_SYSTEM altında tanımlıdır.
- PING ve PONG command payload'u 4-byte Ping ID içerir ve Length alanı 0x04'tür.
- PING yalnız unicast olarak geçerlidir; Broadcast PING'e PONG üretilmez.
- Bir cevabın Source Node değeri, cevabı üreten mantıksal node kimliği olmalıdır.